

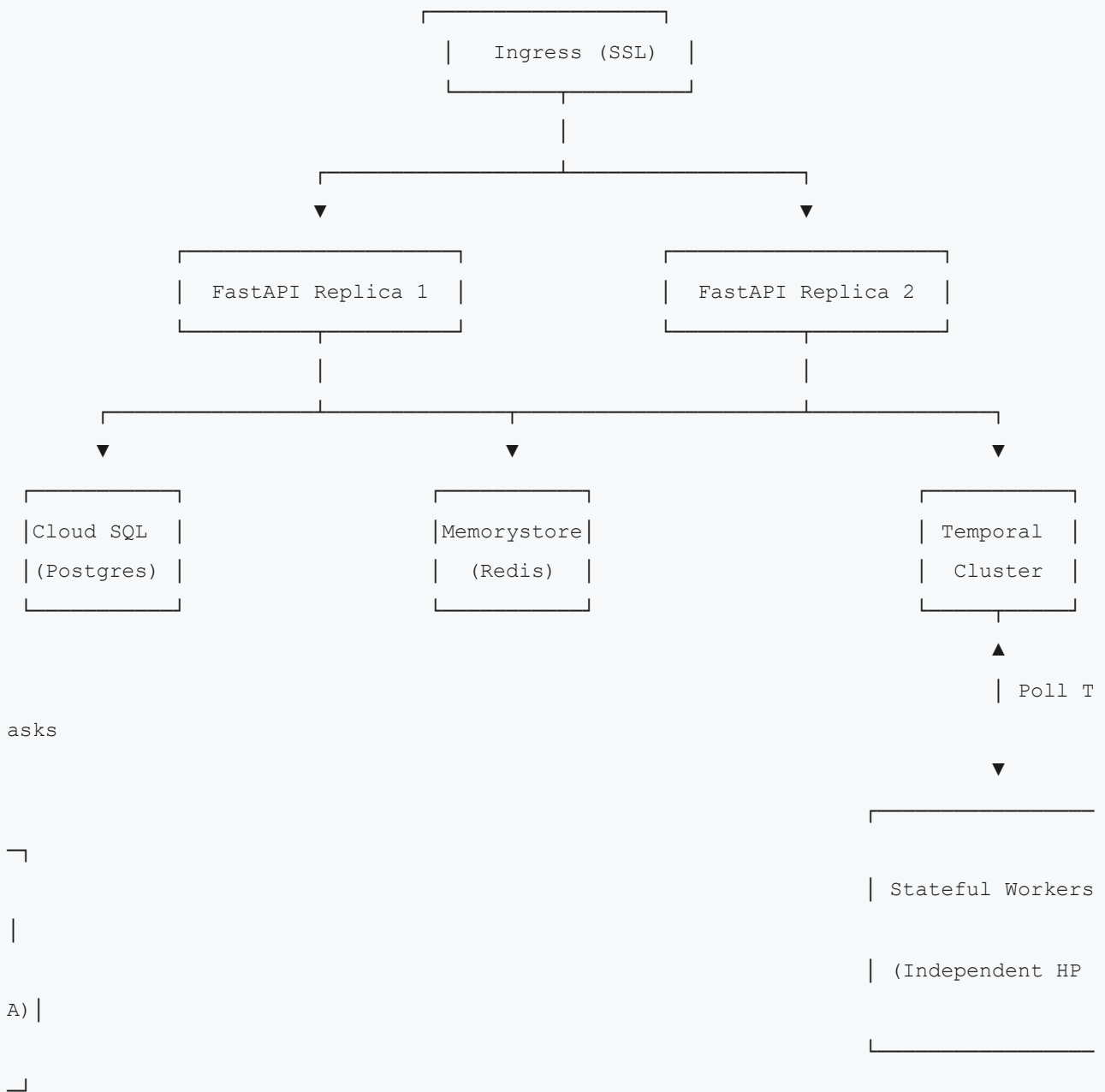
PROJECT 31A — PLATFORM DEPLOYMENT BIBLE (DOCUMENT 13)

Version 1.0.0

Classification: CONFIDENTIAL — FOR INTERNAL DEVELOPMENT AND DUE DILIGENCE ONLY

1. PRODUCTION DEPLOYMENT ARCHITECTURE

Project 31A is deployed as a set of stateless API instances and stateful task workers. Local development utilizes Docker Compose, while staging and production environments run on **Google Kubernetes Engine (GKE)** or equivalent container platforms.



2. SYSTEM PREREQUISITES

Before deploying Project 31A to any environment, ensure the following tools and services are available:

- **Containerization:** Docker 24+ & Docker Compose v2.20+.
- **Language Runtimes:** Python 3.12+ (for backend and workers) and Node.js 20+ (for frontend app).
- **Relational Storage:** PostgreSQL 16+.
- **In-Memory Cache:** Redis 7+.
- **Event Streaming:** Apache Kafka 3.6+ & ZooKeeper 3.8+.
- **Workflow Engine:** Temporal Cluster 1.24+.
- **Vector Search Engine:** Qdrant v1.9+.

- **Object Storage:** MinIO or S3-compatible storage bucket.
-

3. LOCAL DEVELOPMENT SETUP

Follow these steps to initialize the application locally:

3.1 Step 1: Environment Configuration

Copy the default environment variables file:

```
cp .env.example .env
```

Open `.env` and verify the values. Key variables include:

- `APP_ENV=development`
- `APP_DEBUG=true`
- `USE MOCK PROVIDERS=true`
- `DEV_AUTH_BYPASS=true`
- `ENCRYPTION_MASTER_KEY=generate_a_random_32_byte_string`
- `DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/seo_platform`

3.2 Step 2: Start Infrastructure Services

Boot up the containerized databases and message queues:

```
docker-compose -f docker-compose.yml up -d
```

Verify that all nine containers are running:

```
docker-compose ps
```

3.3 Step 3: Run Database Migrations

Initialize the PostgreSQL schema using Alembic:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
```

3.4 Step 4: Boot the Backend API

Start the FastAPI server:

```
uvicorn seo_platform.main:app --host 0.0.0.0 --port 8000 --reload
```

Verify the API is running by hitting the health check endpoint: `curl http://localhost:8000/api/v1/health`

3.5 Step 5: Start the Temporal Workers

In a new terminal window, activate the virtual environment and start the task workers:

```
source .venv/bin/activate  
python -m seo_platform.worker
```

3.6 Step 6: Start the Frontend Application

Navigate to the frontend directory and start the Next.js dev server:

```
cd ../frontend  
npm install  
npm run dev
```

Open your browser and navigate to `http://localhost:3002`.

4. ENVIRONMENT VARIABLES REFERENCE

Variable Name	Required	Default Value	Description
<code>APP_ENV</code>	Yes	<code>development</code>	Runtime mode: <code>development</code> or <code>production</code> .
<code>APP_DEBUG</code>	Yes	<code>true</code>	Enables tracebacks in responses. Must be <code>false</code> in production.
<code>USE MOCK PROVIDERS</code>	Yes	<code>true</code>	Emulates Ahrefs/Hunter calls. Must be <code>false</code> in production.
<code>DEV_AUTH_BYPASS</code>	Yes	<code>true</code>	Skips Clerk JWT checks. Must be <code>false</code> in production.
<code>ENCRYPTION_MASTER_KEY</code>	Yes	None	32-byte key used to encrypt the credentials vault.
<code>DATABASE_URL</code>	Yes	None	Async connection string (must use <code>asyncpg</code>).
<code>REDIS_URL</code>	Yes	None	Redis connection string (e.g. <code>redis://localhost:6379/0</code>).
<code>KAFKA_BOOTSTRAP_SERVERS</code>	Yes	<code>localhost:9092</code>	Comma-separated list of Kafka broker endpoints.
<code>TEMPORAL_HOST</code>	Yes	<code>localhost:7233</code>	Temporal cluster endpoint.
<code>QDRANT_HOST</code>	Yes	<code>localhost:6333</code>	Qdrant vector database endpoint.
<code>MINIO_ENDPOINT</code>	Yes	<code>localhost:9000</code>	Object storage endpoint (replaces S3 in dev).

5. DOCKER IMAGE BUILD DETAILS

The backend application contains a multi-stage `Dockerfile` located in `backend/Dockerfile` to optimize image sizes and security properties.

- **Base Image:** `python:3.12-slim-bookworm` (minimizes vulnerability footprints compared to standard Debian).
- **Build Stage:**
 1. Installs compilation headers (`build-essential`, `libpq-dev`).
 2. Compiles wheel dependencies.
- **Runtime Stage:**
 1. Copies wheel binaries and installs them via `pip`.
 2. Copies application source code to `/app`.
 3. Declares user `appuser` with group `appgroup`.

4. Runs container process as a **non-root user** (`USER 10001`) to protect host directory structures in the event of pod breakouts.

- **Entrypoint:** `uvicorn seo_platform.main:app --host 0.0.0.0 --port 8000 --workers 4`

6. PRODUCTION STABILITY & SYSTEM VERIFICATIONS

6.1 Database Schema Integrity Verifier

During boot, the lifespan context executes a database structure validation. If columns, tables, or native enum types are missing from PostgreSQL, it raises an error. Production API instances will refuse to start if migrations are out-of-sync.

6.2 Health Check Output Schema

The endpoint `/api/v1/health` returns status details of the connected infrastructure:

```
{
  "success": true,
  "data": {
    "status": "healthy",
    "services": {
      "database": "connected",
      "redis": "connected",
      "kafka": "connected",
      "temporal": "connected"
    }
  },
  "error": null,
  "meta": {
    "version": "1.0.0"
  }
}
```

If a service (e.g. Kafka) drops, the status changes to `unhealthy` and returns HTTP 503, signaling to Kubernetes to restart the affected API pod.